Week 1	**Software Engineering**	• Requirement Analysis • SRS • Use-Case Modeling	• Identify requirements for Student, Alumni, Mentor and Admin users. • Prepare use cases for registration and portal access. • Document functional and non-functional requirements.	• Provides a clear development plan. • Reduces ambiguity and requirement changes. • Defines what the system must accomplish.
	OOP	• Classes & Objects • Abstraction	• Identify classes such as `Student`, `Alumni`, `Admin` and `User`. • Represent portal entities using objects.	• Provides modular program structure. • Makes the application easier to maintain and extend.
	DBMS	• ER Model • Entities & Relationships • Database Connectivity	• Design entities for users and registration information. • Establish relationships between database tables. • Connect application with database.	• Creates a proper database foundation. • Ensures registration data can be stored and retrieved reliably.
Week 2	**DBMS**	• ER Diagram • Primary/Foreign Keys • Constraints • SQL	• Create User table and relationships. • Define unique user IDs and required fields. • Use SQL for storing and retrieving user information.	• Maintains data integrity. • Prevents duplicate and invalid user records.
	OOP	• Encapsulation • Classes & Objects	• Keep user credentials and authentication operations inside appropriate classes.	• Protects data and provides controlled access to user information.
	Java	• Methods• Exception Handling• JDBC	• Implement login methods and database connectivity.	• Provides reliable authentication and database interaction.

			• Handle invalid credentials and database errors.	
	Cybersecurity	• Authentication • Password Hashing • Access Control	• Authenticate registered users and securely store passwords.	• Protects user accounts and prevents unauthorized access.
Week 3	**DBMS**	• CRUD Operations • UPDATE • Constraints	• Retrieve profile information and update alumni/student details using SQL.	• Enables accurate profile management and maintains database consistency.
	OOP	• Encapsulation • Methods • Object Interaction	• Create profile-related classes and methods such as `viewProfile()` and `updateProfile()`.	• Makes profile functionality organized, reusable and maintainable.
	Java	• Collections • Exception Handling	• Manage profile data and handle invalid profile updates.	• Improves reliability and simplifies data management.
	Software Testing	• Integration Testing • Regression Testing • End-to-End Testing	• Test Registration → Login → Profile workflow.	• Ensures that connected modules work correctly together.
Week 4	**Data Structures**	• Arrays • Linear Search • Binary Search • Time Complexity	• Search alumni records using Linear Search or Binary Search depending on data arrangement.	• Provides efficient alumni searching and demonstrates algorithmic efficiency.
	Algorithms	• Searching Algorithms • Complexity Analysis	• Compare search approaches based on dataset size and ordering.	• Helps select an appropriate search algorithm and improve performance.

	DBMS	• SELECT • WHERE • Indexing	• Retrieve alumni records based on name, department, batch, skills, etc.	• Enables accurate and faster retrieval of alumni information.
Week 5	**DBMS**	• Table Design •INSERT/UPDATE • Primary/Unique Keys	• Create a Mentor Request table and store student-alumni requests.	• Maintains request records and prevents duplicate requests.
	OOP	• Classes & Objects • Encapsulation	• Create a MentorRequest class with request details and operations.	• Makes mentoring functionality modular and easy to modify.
	Data Structures	• Lists • Sets	• Maintain collections of mentor requests and identify duplicate requests.	• Enables efficient request management and duplicate detection.
	Java	• Collections • Exception Handling	• Use collections to manage requests and handle invalid operations.	• Improves application reliability and data processing.
Week 6	**OOP**	• Encapsulation • Polymorphism • State-based Behavior	• Model request states such as Pending, Accepted and Rejected. • Implement operations for different request states.	• Provides organized workflow management and easier future enhancement.
	DBMS	• UPDATE • Transactions • Data Consistency	• Update mentor request status after Accept/Reject actions.	• Ensures that status changes are correctly and consistently stored.
	Data Structures	• Lists • Maps	• Maintain and retrieve pending/accepted/rejected requests.	• Makes request retrieval and status management efficient.